

Task Management System Using JavaScript and JSON Server

1. Project Overview

Develop a web-based Task Management System using HTML, CSS, JavaScript, Fetch API, and JSON Server.

The application allows users to create, view, update, search, filter, and delete tasks. JSON Server will act as a lightweight REST API and store task information inside a `db.json` file.

The application should provide a responsive dashboard that displays task statistics, a task form, search and filtering options, and a task listing table.

2. Business Scenario

An organization needs a simple task-tracking application to manage day-to-day work.

Employees and team members should be able to record tasks, assign priorities, specify due dates, update task progress, and remove completed or unnecessary tasks.

The organization does not currently require a full backend application. Therefore, JSON Server will be used as a mock REST API for development and training purposes.

3. Technology Stack

- HTML5
 - CSS3
 - JavaScript
 - Fetch API
 - JSON Server
 - REST API
 - JSON
 - Visual Studio Code
 - Live Server
 - Postman
-

4. Task Details

Each task should contain the following information:

Field	Description
ID	Unique task identifier
Title	Short title of the task
Description	Detailed task description
Assigned To	Person responsible for the task
Category	Task category
Priority	Low, Medium, High, or Critical
Status	Pending, In Progress, Completed, or Cancelled
Start Date	Date on which work begins
Due Date	Expected completion date
Created At	Date and time when the task was created

5. Functional Requirements

5.1 Create Task

The user should be able to enter task details and submit the form.

The application should send a POST request to:

```
http://localhost:3000/tasks
```

After successful creation:

- Display a success message.
- Clear the form.
- Refresh the task list.
- Update dashboard statistics.

5.2 View All Tasks

The application should retrieve all tasks using a GET request:

```
GET http://localhost:3000/tasks
```

The task list should display:

- Task title
 - Assigned person
 - Category
 - Priority
 - Start date
 - Due date
 - Status
 - Edit action
 - Delete action
-

5.3 View Task by ID

A single task should be retrievable using:

```
GET http://localhost:3000/tasks/{id}
```

Example:

```
GET http://localhost:3000/tasks/1
```

5.4 Update Task

When the user clicks the Edit button:

- Populate the form with the existing task details.
- Change the form heading to "Update Task."
- Change the submit button text to "Update Task."
- Display a Cancel Update button.

The application should send a PUT request:

```
PUT http://localhost:3000/tasks/{id}
```

After successful update:

- Display a success message.
- Reset the form.
- Refresh the task list.
- Update statistics.

5.5 Delete Task

When the user clicks the Delete button:

- Display a confirmation modal.
- Show the selected task title.
- Allow the user to cancel or confirm deletion.

The application should send a DELETE request:

```
DELETE http://localhost:3000/tasks/{id}
```

After successful deletion:

- Close the confirmation modal.
- Display a success message.
- Refresh the task list.
- Update dashboard statistics.

5.6 Search Tasks

The user should be able to search tasks using:

- Task title
- Description
- Assigned person
- Category
- Priority
- Status

Search results should update dynamically while the user types.

5.7 Filter Tasks

The user should be able to filter tasks based on:

- All statuses
- Pending
- In Progress
- Completed
- Cancelled

The user should also be able to filter tasks based on priority.

6. Dashboard Requirements

The application dashboard should display:

- Total Tasks
- Pending Tasks
- In Progress Tasks
- Completed Tasks

The statistics should update whenever a task is created, updated, or deleted.

7. Form Validation Requirements

The application should validate the following fields:

Task Title

- Required
- Minimum three characters

Description

- Required
- Minimum ten characters

Assigned To

- Required
- Minimum three characters

Category

- Required

Priority

- Required

Status

- Required

Start Date

- Required

Due Date

- Required
- Must not be earlier than the start date

Validation messages should appear below the corresponding fields.

8. Task Categories

The form should provide the following categories:

- Development
 - Testing
 - Documentation
 - Training
 - Meeting
 - Support
 - Research
 - Other
-

9. Task Priorities

The supported priorities are:

- Low
- Medium
- High
- Critical

Each priority should use a different visual badge.

10. Task Statuses

The supported task statuses are:

- Pending
- In Progress
- Completed
- Cancelled

Each status should use a different visual badge.

11. REST API Operations

Operation	HTTP Method	Endpoint
Get all tasks	GET	/tasks
Get task by ID	GET	/tasks/{id}
Create task	POST	/tasks
Replace task	PUT	/tasks/{id}
Partially update task	PATCH	/tasks/{id}
Delete task	DELETE	/tasks/{id}

Base URL:

```
http://localhost:3000
```

12. Suggested Project Structure

```
task-management-app/  
|  
├─ index.html  
├─ db.json  
├─ package.json  
|  
├─ css/  
|   └─ style.css  
|  
└─ js/  
    └─ app.js
```

13. Sample JSON Structure

```
{  
  "tasks": [  
    {  
      "id": "1",  
      "title": "Develop Login Page",  
      "description": "Create a responsive login page with client-side
```

```
validation.",
  "assignedTo": "Amit Sharma",
  "category": "Development",
  "priority": "High",
  "status": "In Progress",
  "startDate": "2026-08-01",
  "dueDate": "2026-08-10",
  "createdAt": "2026-08-01T10:30:00"
}
]
}
```

14. Acceptance Criteria

The application will be considered complete when:

1. Users can create a valid task.
2. All tasks are retrieved from JSON Server.
3. Tasks are displayed in a responsive table.
4. Users can edit an existing task.
5. Users can delete a task after confirmation.
6. Search works without reloading the page.
7. Status and priority filtering work correctly.
8. Dashboard statistics are updated dynamically.
9. Validation messages are displayed correctly.
10. The application works on desktop, tablet, and mobile screens.
11. API failures display meaningful error messages.
12. The application is executed using Live Server and JSON Server.

15. Expected Learning Outcomes

After completing this project, participants should be able to:

- Build responsive user interfaces using HTML and CSS.
- Work with JavaScript events and DOM manipulation.
- Use the Fetch API.
- Perform CRUD operations.
- Work with REST endpoints.
- Use async and await.
- Handle API errors.
- Validate HTML forms.
- Search and filter JavaScript arrays.
- Use JSON Server as a mock backend.